

1 Data Generation

1.1 Pattern Matching

A *pattern* is a set.

1.1.1 Example

Let $a \circ b = [a_1, \dots, a_n, b_1, \dots, b_m]$, where $a = [a_1, \dots, a_n]$ and $b = [b_1, \dots, b_m]$.

$$\begin{aligned}\mathfrak{E} &:= \mathfrak{T} \cup \{\mathfrak{e}_1 \circ \mathfrak{o} \circ \mathfrak{e}_2 \mid \mathfrak{o} \in \{["+"], ["-"]\} \wedge \mathfrak{e}_1, \mathfrak{e}_2 \in \mathfrak{E}\} \\ \mathfrak{T} &:= \mathfrak{F} \cup \{\mathfrak{t}_1 \circ \mathfrak{o} \circ \mathfrak{t}_2 \mid \mathfrak{o} \in \{["\times"], ["\div"]\} \wedge \mathfrak{t}_1, \mathfrak{t}_2 \in \mathfrak{T}\} \\ \mathfrak{F} &:= \mathfrak{N} \cup \{["("] \circ \mathfrak{e} \circ [")"] \mid \mathfrak{e} \in \mathfrak{E}\} \\ \mathfrak{N} &:= N \cup N^* \\ N &:= \{ "0", \dots, "9" \}\end{aligned}$$

What is the difference between pattern matching and tokenwise parsing?

2 gen.jl and pattern.jl

A *grimoire* of a pattern $\mathfrak{P}$ is a function $\Phi : L_\Phi \rightarrow \mathcal{P}(\mathfrak{P})$, where $L_\Phi \subseteq \mathcal{P}(\mathfrak{P})$ is a set of *labels*¹.

Let $G := (\langle \mathcal{G} \rangle, *)$ be a semigroup. Let $\Delta : \boxed{\Phi} \times G^2 \rightarrow \boxed{\Phi}$ – where $\boxed{\Phi}$ is the set of all grimoires of a pattern $\mathfrak{P} \subseteq G^{<\omega}$ – be defined for some grimoire Φ by $\forall l \in L_\Phi \setminus \{l_1 * l_2\} \ (\Phi'(l) = \Phi(l))$ and $\Phi'(l_1 * l_2) = \Phi(l_1 * l_2) \cup \{[l_1, l_2]\}$ if $l_1 * l_2 \in L_\Phi$ and $\Phi'(l_1 * l_2) = \{[l_1, l_2]\}$ if not – where $\Phi' = \Delta(\Phi, [l_1, l_2])$ and $l_1, l_2 \in G$. Note that since $L_\Phi = \text{dom}(\Phi)$, then $L_{\Phi'} = L_\Phi \cup \{l_1 * l_2\}$ if $\Phi' = \Delta(\Phi, [l_1, l_2])$.

Now, consider the exploration function $\mathcal{E} : \boxed{\Phi} \times \boxed{\mathcal{C}}^{<\omega} \rightarrow \boxed{\Phi}$ defined by $\mathcal{E}(\Phi, [\mathcal{C}_1, \dots, \mathcal{C}_n]) = \bigcirc_{i=1}^n \Delta(\Phi, \mathcal{C}_i(L_\Phi))$, where $\boxed{\mathcal{C}} \subset \mathcal{P}(G) \rightarrow G^2$ and $\forall \mathcal{C} \in \boxed{\mathcal{C}} \ (\forall S \in \mathcal{P}(G) \ (\mathcal{C}(S) \in S))$. Then Φ_n is a partial or full representation of the group G , where each element in $L(\Phi_n)$ maps to the set of ordered pairs of elements in G to which it is mapped by $*$.

therefore, for each output generated by $\mathcal{E}$, one can define a function $\mathcal{Z}$ such that: Let a function $\mathcal{Z} : \boxed{\Phi} \times G^{<\omega} \times \boxed{\Pi} \rightarrow \mathcal{G}^{<\omega}$ be defined by

$$\mathcal{Z}(\Phi, \sigma, \Pi) = \begin{cases} \sigma & : \forall l \in \sigma \ (l \in \mathcal{G}) \\ \mathcal{Z}_{\text{step}}(\Phi, \sigma, \Pi), \Pi & : \text{otherwise} \end{cases},$$

where $\mathcal{Z}_{\text{step}}(\Phi, \sigma, \Pi) = \bigotimes_{i=1}^{|\sigma|} \Pi(\Phi(\sigma_i))$ and $\boxed{\Pi} \subset \mathcal{P}(G^{<\omega}) \rightarrow G^{<\omega}$ where $\forall \Pi \in \boxed{\Pi} \ (\forall S \in \mathcal{P}(G^{<\omega}) \ (\Pi(S) \in S))$.

¹Labels – or, *concepts* – are useful abstractions of patterns, which will be used later to optimize the complexity of solution-finding systems. The subsets $\mathfrak{T}, \mathfrak{F}, \mathfrak{N}$ and N in example 1.1.1 are examples of labels.

For each $l \in L_{\Phi_n}$, then, a set of vectors $\mathbf{s} \in \mathcal{G}^{<\omega}$ can be inferred with $\lambda: \boxed{\Phi} \times G \rightarrow \mathcal{G}^{<\omega}$!!!!!!!!! should $\mathcal{G}$ be included in domain too? probably, yes!!!!!!!, defined by

$$\lambda(\Phi, g) = \begin{cases} g & : g \in \mathcal{G} \\ \bigcup_{k \in \Phi(g)} \left(\bigotimes_{j \in k} \lambda(\Phi, j) \right) & : \text{otherwise} \end{cases}$$

!!!!!!!!!!!!THE DEFINITION IS WRONG, NOW THE VECTORS WILL INCLUDE SETS OF POSSIBLE VECTORS!!!!!!!!!!!!

SKRIV TILL RÄTTA DEFINITIONEN AV λ , SÅSOM STÅR I ANTECKNINGSBOKEN. BEVISA FAKTUM OM DEN GENERERADE STRUKTUREN.